

PEC4

Nombre: José Carlos López Henestrosa

Declaro que ostento la autoría total y llena de todas las tareas que se llevan a cabo en el presente documento. Soy la única persona que ha elaborado cada ejercicio. No he compartido los enunciados con nadie y la única ayuda que he recibido ha sido a través del aula de la UOC y su profesorado y he citado de forma explícita los recursos externos que he usado en la preparación de la PEC.

Todas las referencias académicas en esta entrega hacen referencia al libro *Introducción al aprendizaje computacional* de los recursos de esta asignatura, salvo que se cite otra fuente explícitamente.

Descripción de la PEC a realizar

Problema 1 (25 %)

Una clínica veterinaria quiere clasificar automáticamente muestras de tejido animal según tres características medidas en laboratorio. Las características son: nivel de proteína (A1), nivel de glucosa (A2) y temperatura de muestra en grados (A3). Las clases posibles son: Tipo X, Tipo Y y Tipo Z.

Conjunto de entrenamiento:

ID	A1	A2	A3	Clase
1	2	3	1	X
2	4	1	2	X
3	6	5	3	Y
4	5	4	5	Y
5	1	6	4	Z
6	3	7	6	Z

Muestra de test: $x = (A1 = 3.0, A2 = 4.0, A3 = 2.0)$

- a) Calcula la distancia euclídea entre la muestra de test $\mathbf{x}$ y cada uno de los 6 ejemplos del conjunto de entrenamiento. Muestra el desarrollo completo de cada cálculo. [10 %]

Para ello, usamos la métrica de la distancia euclídea, que se define como $de(x, x_i) = \sqrt{\sum_{j=1}^m (x_j - x_{ij})^2}$. A partir de esta, calculamos la distancia entre la muestra de test y cada uno de los ejemplos del conjunto de entrenamiento:

- **ID 1:** $de(x, x_1) = \sqrt{(3 - 2)^2 + (4 - 3)^2 + (2 - 1)^2} = \sqrt{1^2 + 1^2 + 1^2} = \sqrt{3} \approx 1.73$

- **ID 2:** $de(x, x_2) = \sqrt{(3-4)^2 + (4-1)^2 + (2-2)^2} = \sqrt{(-1)^2 + 3^2 + 0^2} = \sqrt{1+9+0} = \sqrt{10} \approx 3.16$
- **ID 3:** $de(x, x_3) = \sqrt{(3-6)^2 + (4-5)^2 + (2-3)^2} = \sqrt{(-3)^2 + (-1)^2 + (-1)^2} = \sqrt{9+1+1} = \sqrt{11} \approx 3.32$
- **ID 4:** $de(x, x_4) = \sqrt{(3-5)^2 + (4-4)^2 + (2-5)^2} = \sqrt{(-2)^2 + 0^2 + (-3)^2} = \sqrt{4+0+9} = \sqrt{13} \approx 3.61$
- **ID 5:** $de(x, x_5) = \sqrt{(3-1)^2 + (4-6)^2 + (2-4)^2} = \sqrt{2^2 + (-2)^2 + (-2)^2} = \sqrt{4+4+4} = \sqrt{12} \approx 3.46$
- **ID 6:** $de(x, x_6) = \sqrt{(3-3)^2 + (4-7)^2 + (2-6)^2} = \sqrt{0^2 + (-3)^2 + (-4)^2} = \sqrt{0+9+16} = \sqrt{25} = 5.00$

- b) Indica qué clase asignarías a la muestra $\mathbf{x}$ usando el clasificador 1NN. Justifica tu respuesta con los resultados del apartado anterior. [5 %]

El algoritmo 1NN busca el ejemplo más cercano a la muestra de test y le asigna la clase de dicho prototipo. Basándonos en los resultados del apartado anterior, la distancia mínima corresponde al ejemplo con el **ID 1** ($\sqrt{3} \approx 1.73$). Dado que la clase del ejemplo 1 es **X**, el clasificador 1NN asignará la clase **X** a la muestra x .

- c) Indica qué clase asignarías a la muestra $\mathbf{x}$ usando el clasificador 3NN. En caso de empate en la votación, razona cómo lo resolverías. [5 %]

En el algoritmo 3NN, la clasificación se realiza buscando el conjunto de los 3 ejemplos más cercanos a la muestra y seleccionando la clase más frecuente entre sus etiquetas. Las tres distancias menores calculadas en el primer apartado son las siguientes:

- (a) ID 1 (Distancia ≈ 1.73) $\rightarrow$ Clase **X**
- (b) ID 2 (Distancia ≈ 3.16) $\rightarrow$ Clase **X**
- (c) ID 3 (Distancia ≈ 3.32) $\rightarrow$ Clase **Y**

Al contar los votos, tenemos dos votos para la clase X y un voto para la clase Y. Por lo tanto, se asignaría la clase **X** a la muestra.

En caso de producirse un empate en la votación, una estrategia válida para resolverlo sería ponderar los votos utilizando la **inversa de la distancia** euclídea, lo cual otorga más peso en la decisión final a los vecinos que estén más cerca de la muestra de test.

- d) El kNN tiene un coste computacional alto en la fase de clasificación cuando el conjunto de entrenamiento es muy grande. Explica por qué ocurre esto y propón una estrategia conceptual para reducir ese coste sin cambiar el algoritmo por otro completamente diferente. [5 %]

Esto ocurre porque el algoritmo kNN, en su forma más simple y según el **Apartado 2.1. k Nearest Neighbors (kNN)**, "deja en la memoria todos los ejemplos durante el proceso de entrenamiento". En consecuencia, la generalización se pospone hasta el momento de clasificar nuevos ejemplos, instante en el que debe calcular la distancia entre la nueva muestra y *todos* los ejemplos almacenados en la base de datos.

Para reducir el coste computacional sin abandonar el algoritmo basado en vecinos más cercanos, se puede aplicar una técnica de **edición o condensación del conjunto de entrenamiento**. Esto consiste en un preprocesamiento que elimina de la memoria los ejemplos redundantes, como aquellos que están en el interior de una clase y no afectan las

fronteras de decisión, conservando únicamente una fracción de los datos más representativos. Otra alternativa estructural es el uso de **árboles de búsqueda espacial, como los árboles KD**, que organizan los datos geoméricamente y permiten descartar bloques enteros de puntos alejados sin necesidad de calcular la distancia a cada uno de ellos.

Problema 2 (25 %)

Un sistema automatizado debe clasificar solicitudes de crédito bancario como Aprobadas (A) o Rechazadas (R) en función de tres atributos categóricos del solicitante.

ID	Empleo	Historial	Ingresos	Class
1	Fijo	Bueno	Alto	A
2	Temporal	Bueno	Medio	A
3	Fijo	Malo	Alto	R
4	Temporal	Malo	Bajo	R
5	Fijo	Bueno	Medio	A
6	Fijo	Malo	Medio	R
7	Temporal	Bueno	Alto	A
8	Temporal	Malo	Alto	R

- a) Calculad la bondad de la partición para cada uno de los tres atributos (Empleo, Historial, Ingresos) usando el criterio de clase mayoritaria descrito en el módulo. Muestra todos los conjuntos generados y el número de ejemplos bien clasificados. [12 %]

Para calcular la bondad de la partición usando el criterio de clase mayoritaria, dividimos el conjunto de entrenamiento según los valores de cada atributo, asignamos la clase más frecuente a cada conjunto, contamos los ejemplos bien clasificados y dividimos entre el número total de ejemplos.

- **Atributo Empleo:**

- *Fijo*: {ID 1 (A), 3 (R), 5 (A), 6 (R)} → Empate (2 A, 2 R). Asignamos una clase cualquiera como mayoritaria. Ejemplos bien clasificados: 2.
- *Temporal*: {ID 2 (A), 4 (R), 7 (A), 8 (R)} → Empate (2 A, 2 R). Asignamos una clase cualquiera como mayoritaria. Ejemplos bien clasificados: 2.

$$\text{Bondad(Empleo)} = \frac{2+2}{8} = \frac{4}{8} = 0.5$$

- **Atributo Historial:**

- *Bueno*: {ID 1 (A), 2 (A), 5 (A), 7 (A)} → Mayoría A (4 A, 0 R). Ejemplos bien clasificados: 4.
- *Malo*: {ID 3 (R), 4 (R), 6 (R), 8 (R)} → Mayoría R (0 A, 4 R). Ejemplos bien clasificados: 4.

$$\text{Bondad(Historial)} = \frac{4+4}{8} = \frac{8}{8} = 1.0$$

- **Atributo Ingresos:**

- *Alto*: {ID 1 (A), 3 (R), 7 (A), 8 (R)} → Empate (2 A, 2 R). Asignamos una clase cualquiera. Ejemplos bien clasificados: 2.

- *Medio*: {ID 2 (A), 5 (A), 6 (R)} → Mayoría A (2 A, 1 R). Ejemplos bien clasificados: 2.
- *Bajo*: {ID 4 (R)} → Mayoría R (0 A, 1 R). Ejemplos bien clasificados: 1.

$$\text{Bondad}(\text{Ingresos}) = \frac{2+2+1}{8} = \frac{5}{8} = 0.625$$

- b) Selecciona el atributo raíz del árbol e indica si alguno de los nodos resultantes ya es terminal. Para los nodos internos, aplica una segunda iteración del algoritmo (sin entrar en una tercera). [8 %]

En cada iteración, el algoritmo selecciona el atributo que hace la mejor partición del conjunto, por lo que el atributo raíz elegido es **Historial**, que obtuvo el valor máximo de bondad de partición (1.0).

Esta partición genera dos nodos derivados de la regla atributo = valor:

- **Nodo 1 (Historial = Bueno)**: Contiene únicamente ejemplos con clase A. Como todos los ejemplos quedan bien clasificados, se convierte en un **nodo terminal** con la clase A.
- **Nodo 2 (Historial = Malo)**: Contiene únicamente ejemplos con clase R. Por la misma razón, se convierte en un **nodo terminal** con la clase R.

Dado que la partición inicial generada por el atributo Historial agrupa todas las muestras en nodos donde la clasificación es exacta, **no se genera ningún nodo interno** y, por tanto, no es necesario aplicar una segunda iteración al conjunto reducido.

- c) Clasifica la siguiente muestra de test usando el árbol construido y justifica el recorrido rama a rama:

Muestra test: Empleo = Fijo, Historial = Malo, Ingresos = Alto

Para hacer el test, exploramos el árbol partiendo de la raíz y escogemos las ramas que se correspondan con los valores de los atributos del ejemplo de test. Con tal de facilitar su desarrollo, procedemos a explicarlo con pasos numerados:

1. El recorrido comienza en el nodo raíz del árbol de decisión construido: **Historial**.
2. Comprobamos el valor del atributo "Historial" en la muestra de test, que en este caso es **Malo**.
3. Descendemos por la rama *Historial = Malo*.
4. Inmediatamente llegamos a un nodo terminal y damos como predicción la clase correspondiente a ese nodo, que es la **clase R (Rechazada)**.

Por tanto, la solicitud sería clasificada como **Rechazada (R)**.

- d) ¿Cuál es la principal ventaja del árbol de decisión respecto al kNN en un contexto bancario real? Razona tu respuesta. [5 %]

La principal ventaja del árbol de decisión es, citando al **Apartado 2.2. Árboles de decisión**, "la facilidad de interpretación del modelo de aprendizaje", ya que proporciona "una información clara de la toma de decisiones y de la importancia de los diferentes atributos involucrados", razón por la cual tienen sentido aplicarlo en el sector bancario.

A diferencia del algoritmo kNN, que simplemente asigna una clase basándose en comparaciones de distancia o similitud pura sin explicar el porqué, un árbol de decisión genera un conjunto de reglas deductivas explícitas de tipo "atributo = valor". Si un banco rechaza la solicitud de crédito de un cliente basándose en kNN, el sistema sólo podría justificarlo

diciendo que el cliente se parece a otras personas que no pagaron. Al usar un árbol de decisión, el banco puede darle un motivo formal, lógico y auditable legalmente. Por ejemplo: "Su solicitud fue rechazada exclusivamente porque su historial crediticio previo es malo, sin importar sus ingresos".

Problema 3 (25 %)

Se dispone de un perceptrón con 2 neuronas de entrada, una capa oculta de 2 neuronas y 1 neurona de salida. La función de activación de las neuronas ocultas es ReLU. La neurona de salida aplica una función lineal. Los pesos y sesgos son fijos y se dan a continuación:

Arquitectura y parámetros:

Capa de entrada $\rightarrow$ capa oculta (pesos w , sesgo b):

- Neurona oculta H1: $w_{11} = 0.5$, $w_{21} = 0.2$, $b_1 = 0.1$
- Neurona oculta H2: $w_{12} = -0.3$, $w_{22} = 0.8$, $b_2 = -0.2$

Capa oculta $\rightarrow$ neurona de salida (pesos v , sesgo b_{out}):

- $v_1 = 0.7$, $v_2 = -0.4$, $b_{\text{out}} = 0.3$

Entrada: $x = (x_1 = 1.5, x_2 = 2.0)$

Salida esperada (etiqueta): $y^* = 1.2$

- a) Calcula la entrada neta y la salida de cada neurona oculta (H1 y H2). Muestra el cálculo de la suma ponderada antes y después de aplicar ReLU. [8 %]

Para calcular la entrada neta z de cada unidad oculta, combinamos las entradas x con su vector de pesos correspondiente y sumamos el sesgo b . A este resultado le aplicamos la función de activación ReLU, que produce una salida de 0 si la entrada neta es negativa y la mantiene intacta si es positiva.

Neurona oculta H1:

- Entrada neta (z_1): $z_1 = (w_{11} \cdot x_1) + (w_{21} \cdot x_2) + b_1$
- Sustituimos: $z_1 = (0.5 \cdot 1.5) + (0.2 \cdot 2.0) + 0.1 = 0.75 + 0.4 + 0.1 = \mathbf{1.25}$
- Salida (h_1): $\text{ReLU}(1.25) = \max(0, 1.25) = \mathbf{1.25}$

Neurona oculta H2:

- Entrada neta (z_2): $z_2 = (w_{12} \cdot x_1) + (w_{22} \cdot x_2) + b_2$
- Sustituimos: $z_2 = (-0.3 \cdot 1.5) + (0.8 \cdot 2.0) - 0.2 = -0.45 + 1.6 - 0.2 = \mathbf{0.95}$
- Salida (h_2): $\text{ReLU}(0.95) = \max(0, 0.95) = \mathbf{0.95}$

- b) Calcula la salida final de la red ($\hat{y}$) a partir de las salidas de H1 y H2. [5 %]

La capa de salida recibe las señales de la capa oculta, h_1 y h_2 , y calcula la entrada neta combinando estos valores con sus propios pesos y sesgo.

Entrada neta (z_{out}): $z_{\text{out}} = (v_1 \cdot h_1) + (v_2 \cdot h_2) + b_{\text{out}}$

Sustituimos los valores calculados: $z_{\text{out}} = (0.7 \cdot 1.25) + (-0.4 \cdot 0.95) + 0.3 = 0.875 - 0.38 + 0.3 = \mathbf{0.795}$

Puesto que el enunciado especifica que la neurona de salida aplica una **función de activación lineal**, la salida no se modifica, ya que se multiplica por 1.

Entonces, la salida final de la red es $\hat{y} = \mathbf{0.795}$.

- c) Calcula el error cuadrático C entre la salida obtenida $\hat{y}$ y la etiqueta y^* . [4 %]

Para la capa oculta, calculamos la entrada neta y aplicamos la función de activación ReLU, la cual se define como $g(z) = \max(0, z)$:

- $z_1 = w_{11}x_1 + w_{21}x_2 + b_1 = 0.5(1.5) + 0.2(2.0) + 0.1 = 0.75 + 0.4 + 0.1 = 1.25$.
La activación es $h_1 = \max(0, 1.25) = 1.25$.
- $z_2 = w_{12}x_1 + w_{22}x_2 + b_2 = -0.3(1.5) + 0.8(2.0) - 0.2 = -0.45 + 1.6 - 0.2 = 0.95$.
La activación es $h_2 = \max(0, 0.95) = 0.95$.

Para la capa de salida con activación lineal calculamos $\hat{y}$:

$$\hat{y} = v_1h_1 + v_2h_2 + b_{\text{out}} = 0.7(1.25) - 0.4(0.95) + 0.3 = 0.875 - 0.380 + 0.3 = 0.795.$$

El error cuadrático C entre la salida obtenida y la esperada se calcula como:

$$\mathbf{C} = (\hat{y} - y^*)^2 = (0.795 - 1.2)^2 = (-0.405)^2 = \mathbf{0.164025}.$$

- d) Supón ahora que el peso w_{12} cambia de -0.3 a $+0.6$ (manteniendo todo lo demás igual). Sin recalcular todos los pasos desde cero, razona cualitativamente: ¿aumentará o disminuirá la activación de H2? ¿Qué efecto tendrá esto en la salida final $\hat{y}$ y en el error C ? [8 %]

El valor de entrada neta z_2 será mayor, ya que la entrada $x_1 = 1.5$ es positiva. Como la función ReLU es igual a z para valores positivos, un valor mayor de z_2 provocará que la activación de la neurona H2 (h_2) **aumente**.

En la capa de salida, el peso asociado a h_2 es $v_2 = -0.4$. Al ser un peso negativo, un aumento en la activación h_2 hará que el valor de la salida final $\hat{y}$ **disminuya**.

Dado que nuestra salida original ($\hat{y} = 0.795$) ya era menor que la salida esperada ($y^* = 1.2$), una disminución adicional en $\hat{y}$ la alejará aún más del objetivo y^* . Como resultado, la magnitud de la diferencia entre el valor esperado y el obtenido será mayor, por lo que el error cuadrático C **aumentará**.

Problema 4 (25 %)

Un sistema de aprendizaje supervisado ha sido entrenado para detectar si una planta tiene una enfermedad (Positivo = enferm, Negativo = sana). Se ha evaluado sobre un conjunto de tests de 200 muestras y se han obtenido los resultados siguientes expresados en la matriz de confusión.

	Positivo	Negativo
Positivo	45	15
Negativo	20	120

- a) Calcula las siguientes métricas mostrando la fórmula, la sustitución numérica y el resultado final: [12 %]

- Exactitud (accuracy)
- Precisión (precision)
- Sensibilidad (recall)
- Medida F1

Para calcular las métricas, primero identificamos los componentes de la matriz de confusión:

- **Verdaderos positivos (tp):** 45
- **Falsos positivos (fp):** 15
- **Falsos negativos (fn):** 20
- **Verdaderos negativos (tn):** 120

Ahora, aplicamos las fórmulas teóricas de evaluación:

1. Exactitud (accuracy): Mide el número de ejemplos clasificados correctamente sobre el total.

$$Accuracy = \frac{tp + tn}{tp + tn + fp + fn} = \frac{45 + 120}{200} = \frac{165}{200} = \mathbf{0.825 \text{ (82.5\%)}}$$

2. Precisión (precision): Mide los ejemplos positivos bien clasificados respecto al total de ejemplos con *predicción* positiva.

$$Precision = \frac{tp}{tp + fp} = \frac{45}{45 + 15} = \frac{45}{60} = \mathbf{0.75 \text{ (75.0\%)}}$$

3. Sensibilidad (recall): Mide los ejemplos positivos bien clasificados respecto al total de ejemplos *reales* positivos.

$$Recall = \frac{tp}{tp + fn} = \frac{45}{45 + 20} = \frac{45}{65} \approx \mathbf{0.6923 \text{ (69.23\%)}}$$

4. Medida F1: Es una combinación de las dos métricas anteriores para balancear sus efectos.

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} = \frac{2 \cdot 0.75 \cdot 0.6923}{0.75 + 0.6923} = \frac{1.03845}{1.4423} \approx \mathbf{0.720 \text{ (72.0\%)}}$$

- b) En el contexto de la detección de enfermedades en plantas, ¿cuál de las cuatro métricas anteriores consideras que debería ser la métrica prioritaria para evaluar este sistema? Argumenta tu elección teniendo en cuenta las consecuencias de los falsos negativos (FN) y los falsos positivos (FP) en este dominio. [7 %]

En la detección de enfermedades, la métrica prioritaria debe ser la **Sensibilidad (recall)**, ya que un **falso negativo** significa clasificar a una planta enferma como sana. Las consecuencias de esto son muy graves, puesto que la planta no recibirá tratamiento y la enfermedad podría propagarse y arruinar a toda la cosecha. Por otro lado, un **falso positivo** implicaría catalogar como enferma a una planta sana, lo cual solo incurre en el coste de aplicar un tratamiento preventivo o aislarla innecesariamente, pero no compromete la cosecha. Como la fórmula de la sensibilidad es $\frac{tp}{tp+fn}$, centrarse en maximizar esta métrica hace que el sistema penalice con dureza los falsos negativos, lo cual conlleva minimizar así el riesgo de pasar por alto a las plantas verdaderamente enfermas.

c) Un investigador propone usar validación cruzada con $k = 8$ en lugar de validación simple (holdout) sobre un conjunto de 400 muestras. Explica: [6 %]

- ¿Cuántas muestras se usarían como test en cada iteración?
- ¿Cuántas veces se entrenaría el modelo en total?
- ¿Por qué la validación cruzada da estimaciones más fiables que la validación simple?

Basado en los protocolos de validación y partición de datos:

- **Muestras de test por iteración:** La validación cruzada consiste en dividir un conjunto en k subconjuntos. Si tenemos 400 muestras y $k = 8$, se crean 8 subconjuntos iguales de $\frac{400}{8} = 50$ muestras. En cada iteración, se utiliza un solo subconjunto como test, por lo que se usarían **50 muestras** para testar y las 350 restantes para entrenar.
- **Veces que se entrena el modelo:** Como el protocolo indica que "se hacen k pruebas utilizando en cada una un subconjunto como test", el modelo será entrenado y evaluado **8 veces** en total, rotando el bloque de validación.
- **Fiabilidad superior:** La validación simple presenta un problema: "según el conjunto de datos que tengamos puede variar mucho el comportamiento del sistema en función de la partición de ejemplos", lo que significa que el rendimiento medido depende en gran medida de la suerte al hacer la división inicial. La validación cruzada soluciona este sesgo calculando la media y la desviación estándar de los resultados de sus múltiples iteraciones cruzadas. Como todos los datos tienen que pasar en algún momento tanto por la fase de entrenamiento como por la de test, se obtiene una estimación del rendimiento general mucho más consistente y fiable.